

FeCo: Boosting Intrusion Detection Capability in IoT Networks via Contrastive Learning

Ning Wang^{ID}, *Member, IEEE*, Shanghao Shi^{ID}, *Student Member, IEEE*, Yimin Chen^{ID}, *Member, IEEE*,
Wenjing Lou^{ID}, *Fellow, IEEE*, and Y. Thomas Hou^{ID}, *Fellow, IEEE*

Abstract—Over the last decade, Internet of Things (IoT) has permeated our daily life with a broad range of applications. However, a lack of adequate security in IoT devices renders IoT systems vulnerable to various network-based cyberattacks, potentially causing severe damage. Recent works have explored using machine learning to build anomaly detection models for defending against such attacks. In this paper, we propose FeCo, a federated-contrastive-learning framework that coordinates in-network IoT devices to jointly learn intrusion detection models. FeCo utilizes federated learning to alleviate users' privacy concerns as participating devices only submit their model parameters rather than raw local data. Compared to previous works, we develop a novel representation learning method based on contrastive learning that is able to learn a more accurate model for the benign class. FeCo significantly improves the intrusion detection accuracy compared to previous works. In addition, we implement a two-step feature selection scheme to avoid overfitting and reduce computation time. Through extensive experiments on the NSL-KDD dataset and the BaIoT dataset, we demonstrate that FeCo achieves as high as 8% accuracy improvement compared to the state-of-the-art and is robust to non-independent and identically distributed (non-IID) data. Our implementation of FeCo on a Raspberry Pi device further confirms the applicability of FeCo for resource-constrained IoT devices.

Index Terms—Intrusion detection system, Internet of Things, contrastive learning.

I. INTRODUCTION

THE last decade has seen an exponential growth of the Internet of Things (IoT) devices. Having achieved the milestone of 12 billion connected devices in 2020, it is estimated that by 2025 there will be more than 30.9 billion IoT devices in the market.¹ IoT starts a new paradigm where billions of smart

devices with embedded computational capability and Internet connectivity can automatically work with minimal human intervention. Due to low cost and versatility, IoT devices are being used in almost all sectors: healthcare, smart cities, agriculture, and transportation, to name a few [2], [3], [4].

However, such pervasiveness also increases the risk of data breaches and cyberattacks. In the past decade, we have seen increased attacks involving IoT devices and IoT systems. Many IoT devices have limited on-device resources such as computing power and memory, which limits the amount and types of security mechanisms that can be implemented in them. Many IoT manufacturers are not security-savvy. In a rush to roll out new products, very often only minimal security features are included, not to mention providing ongoing support or software security updates. The default configuration of an IoT device usually remains in place if no one makes an effort to change it [5]. One notable attack on IoT is Mirai [6], which overwhelmed several high-profile targets with massive distributed denial-of-service (DDoS) attacks in late 2016. More than half a million devices were infected in a few months. Security patching is one possible remedy to security issues. However, many devices lack appropriate facilities for automated security updates, or there may be significant delays until device manufacturers provide them. Considering that IoT devices typically connect to the Internet through a local gateway, a more practical and effective idea to secure IoT devices and systems is to implement an IDS in the local gateway. An IDS continuously monitors incoming and outgoing data streams generated by diverse sources and analyzes them to detect cyber threats.

Ideally, a network IDS device should be placed at a data concentration point in the network for best performance. For instance, most often an IDS device is deployed behind the firewall at the gateway of an edge network. For an IDS in an IoT network, there are two main placement strategies: distributed and centralized. In a distributed placement [7], [8], a local gateway or edge router independently manages its own IDS for the local network. Due to the scarcity of local data, distributed IDS may suffer low accuracy. On the other hand, the rising concern of privacy poses great challenges to a centralized placement [9], [10]. Legal restrictions (e.g., HIPAA²) actually prohibit collecting and storing certain types of user data to a central server. In our pursuit to design an efficient, accurate, yet privacy-conscious IDS for IoT network, we resort to the Federated Learning

Received 7 May 2022; revised 14 April 2023; accepted 11 February 2025. Date of publication 20 February 2025; date of current version 11 July 2025. This work was supported in part by the Office of Naval Research under Grant N00014-24-1-2730 and Grant N00014-19-1-2621, in part by the National Science Foundation under Grant 2312447, Grant 2247560, Grant 2154929, Grant 2332675, Grant 2331936, and Grant 2235232, and in part by Virginia Commonwealth Cyber Initiative (CCI). (Corresponding author: Ning Wang.)

Ning Wang is with the Department of Computer Science and Engineering, University of South Florida, Tampa, FL 33620 USA (e-mail: ning18@vt.edu).

Y. Thomas Hou is with the Department of Electrical and Computer Engineering, Virginia Tech, Blacksburg, VA 24061 USA (e-mail: thou@vt.edu).

Shanghao Shi and Wenjing Lou are with the Department of Computer Science, Virginia Tech, Blacksburg, VA 24061 USA (e-mail: shanghaos@vt.edu; wjlou@vt.edu).

Yimin Chen is with the Department of Computer Science, University of Massachusetts Lowell, Lowell, MA 01854 USA (e-mail: ian_chen@uml.edu). Digital Object Identifier 10.1109/TDSC.2025.3544106

¹<https://www.statista.com/statistics/1101442/iot-number-of-connected-devices-worldwide/>

²<https://www.hhs.gov/hipaa/index.html>

(FL) framework [11], [12], [13]. FL enables local gateways to cooperatively contribute to the training of a global model by providing their local model parameter to a central server. Through an iterative learning process, the global model achieves good generalization by learning knowledge from a large number of IoT devices. However, due to device heterogeneity (in terms of communication protocols and co-existing technologies), the pattern of normal network traffic varies dramatically among different types of IoT devices. Learning a universal model across all different device types may render the IDS useless. To address this problem, we choose the device-type-specific design [5] that builds an IDS model for each type of IoT devices, which tends to provide more accurate models.

In general, there exist two main approaches for intrusion detection: anomaly-based Intrusion Detection System (IDS) and signature-based IDS [14], [15], [16], [17]. An anomaly-based IDS learns a model that represents the normal behavior and generates an alarm when the deviation of an incoming instance from the normal behavior surpasses a security threshold. A signature-based IDS detects intrusions by comparing incoming traffic with the saved signatures in a database of known attacks. Due to the nature of their design, signature-based IDSs are not able to detect zero-day attacks. We focus on anomaly-based IDS in this paper due to its superior capability of detecting novel attacks. With the great success of machine learning in pattern recognition, utilizing machine learning in anomaly-based IDSs is a significant trend in the last two decades.

While being able to detect novel attacks, the machine-learning-based IDSs also suffer from the high false-positive rate (FPR) compared to the signature-based IDSs. Pajouh et al. [18] proposed to use the Naive Bayes classifier to identify anomalous behavior. Then a K-Nearest Neighbor model is used to further detect anomalies from those instances that are classified as normal by the Naive Bayes model in order to reduce false negatives. Wang et al. [19] combined multi-layer perceptron (MLP) and manifold learning to improve the detection rates. Most current machine-learning-based IDS primarily focus on improving the distinguishing capability of classification models over the original data, which achieves limited improvement when benign data is not easily separable from malicious data in the original feature space. To reduce false positives, we propose to learn new representations by utilizing a contrastive-learning-based mechanism, rather than solely focusing on the classification algorithm. During representation learning, we can actively increase the distance between the benign records and intrusion classes in the embedding space, which significantly improves subsequent detection performance.

Base on the intuition that contrastive learning [20], [21], [22] can actively learn a consistent representation for a group of images with similar foreground but different backgrounds (e.g., a bunch of cat images with different backgrounds) in the computer vision field, we propose to use contrastive learning to learn the essential characteristics of benign network events while ignoring the variance caused by other factors, e.g., device usage pattern. Specifically, we build a feed-forward artificial neural network (ANN) that takes an original traffic instance as input and outputs a new representation. Our goal is to learn a new feature space

where the benign representations lie in a small cluster while attack representations stay far from the benign cluster so that the differentiation of the two can be made easier. By minimizing the volume of a hyper-sphere that encloses the representations of the normal data, we can train an ANN model that is able to extract the common properties of benign variations more precisely, which leads to improved robustness of the normal profile and significantly reduced FPR. Furthermore, in order to reduce the computation overhead, we build a lightweight ANN with only two hidden layers for resource-constrained IoT devices.

Finally, we build **FeCo**, a **Federated-Contrastive-learning** framework, by incorporating FL into the contrastive-learning-based IDS to achieve accurate detection and preserve data privacy simultaneously. In the FeCo design, each local gateway manages a local IDS model and all gateways cooperatively work with a central server to boost local detection performance. To avoid the overfitting of an IDS model to irrelevant features, we propose a two-step feature selection scheme for pre-processing the input data. We remove less significant features and only retain essential information for detection. We extensively evaluate FeCo using a network traffic dataset (i.e., NSL-KDD dataset [23]) to demonstrate the effectiveness of FeCo in detecting intrusions. Our contributions are summarized as follows:

- We propose a novel method for building the “norm” in an anomaly-based IDS by learning new representations for network traffic based on contrastive learning. With the proposed new method, the learned representations of benign inputs lie only in a small cluster, enabling FeCo to extract a stable template for benign inputs. Extensive evaluation results show that representation learning in FeCo significantly boosts its detection accuracy compared to previous works.
- We propose a two-step feature selection scheme to reduce the risk of overfitting. Our feature selection scheme exploits feature correlation and importance and extracts only the essential information for intrusion detection. Such a scheme also helps reduce computation complexity as a result of smaller input dimensionality, making FeCo more suitable for resource-constrained IoT devices.
- We extensively evaluate FeCo using the NSL-KDD dataset and the BaIoT dataset. By comparing FeCo with 11 baselines, we demonstrate the effectiveness of contrastive-learning-based IDS. On the NSL-KDD dataset, FeCo achieves an 8% accuracy improvement over the state-of-the-art. For zero-day attacks (i.e., attacks unseen by the training dataset), FeCo achieves a recall 8% to 42% higher than other baselines. FeCo is robust to non-IID (Independent and Identically Distributed) data by showing consistent accuracy in different data distributions. We also investigate FeCo on the convergence performance, scalability, and overhead to show that it is suitable for IoT systems.
- We implement FeCo on a Raspberry Pi device to demonstrate its applicability in resource-constrained IoT devices. We perform model optimization using model weights quantization technique in order to save memory and storage

space. The optimized light-weight model achieves comparable accuracy with a standard FeCo model.

The remainder of this paper is organized as follows. We introduce the related work in Section II. In Section III, we describe the system model and threat model. The detailed design of FeCo, including feature selection and detection algorithm, is depicted in Section IV. Section V presents the evaluation results using two real-world network traffic datasets. In Section VI, we implement FeCo on a Raspberry Pi to demonstrate its applicability. Finally, Section VII concludes the paper.

II. RELATED WORK

We focus on anomaly-based IDSs in this paper. The key component of a general anomaly-based IDS is a model that can represent the legitimate traffic. In what follows, we review the anomaly-based IDSs with a focus on the IDSs in the domain of IoT.

IDS placement is an important design choice in IoT systems compared to computer networks. There exist three IDS placement strategies: distributed IDS placement [7], [8], centralized IDS placement [9], and hybrid IDS placement [10]. Recently, with the advances of Federated Learning (FL) [11], FL-based IDSs [5], [42], [43] are becoming increasingly popular. FL allows a distributed placement to have a better generalization performance as it takes advantage of diverse sets of training data from a large number of IoT devices. FL also provides better privacy-preservation in IDSs compared to centralized placement. Nguyen et al. [5] are the first to employ FL in anomaly-based IDSs. In their design, a local gateway uses its local data to train the model and submits the model parameters to the cloud server. The cloud server then aggregates these local models into a global model. Since then, multiple works (e.g., [42], [43]) have explored the use of FL framework to enable decentralized edge devices to learn an anomaly detection model using only on-device data at each edge device.

In the area of anomaly-based IDSs, machine learning mechanisms have been extensively researched in the literature. The most popular strategy for detecting attacks is to monitor a network's activity and report potential abnormal events: deviations from profiles of normality previously learned from benign traffic [5], [24], [44], [45]. Du et al. [45] proposed DeepLog that utilizes Long Short-Term Memory to model a system log as a natural language sequence. DeepLog automatically learns log patterns from normal execution and detects anomalies when log patterns deviate from the learned pattern. Mirsky et al. [24] proposed to utilize an ensemble of multiple autoencoders to differentiate between normal and abnormal traffic patterns. The autoencoder reconstructs an input and computes the reconstruction error in terms of root mean squared errors (RMSE). An alarm is generated when RMSE value exceeds a threshold. The work in [46] exploited a non-parametric density estimation method to learn and predict legitimate access pattern. Nguyen et al. [5] modeled network packets as symbols in a language enabling the use of Gated Recurrent Units (GRU) for anomaly detection. Specifically, the GRU model estimates a probability of the next symbol, and an alarm is raised if the occurrence probabilities of a sufficient number of packets fall below a detection threshold.

Besides the mechanisms discussed above, some other IDSs directly learn a binary or multi-class classifier for detecting intrusions. Yan et al. [47] applied SVM to detect botnets using the high-level features extracted from the command and control channel. Pajouh et al. [18] proposed TDTC that utilizes two tiers of classification to improve detect rate. TDTC first uses the Naive Bayes classifier to identify anomalous behavior. Then a K-Nearest Neighbor model is used to further detect anomalies from those instances that are classified as normal by the Naive Bayes model in order to reduce false negatives. Wang et al. [19] employed multi-layer perceptron (MLP) and manifold learning to detect evasive intrusions. Rathore et al. [25] proposed ESFCM that integrates a Fuzzy C-Means with the Extreme Learning Machine (ELM) classifier to achieve efficient attack detection in IoT. Wang et al. [48] proposed a graphic model that stores known traffic patterns as a relational graph between patterns and their labels (malicious or normal) to detect DDoS attacks in the cloud computing scenario.

Most of anomaly-based IDS suffers from a high false-positive rate (FPR) compared to the signature-based IDS. We propose a method, namely the contrastive-learning-based anomaly detection method, to address this problem. Similar to the traditional anomaly-based IDS, the proposed method also aims to learn the pattern of benign traffic. The critical difference is that we maximized the distance from the normal pattern to the intrusion traffic and minimized the distance among normal instances during the pattern learning phase. It then detects network traffic that deviates from the pattern as an intrusion. Our methods can reduce the FPR and maintain the capability to detect novel attacks.

We also summarize representative papers on machine-learning-based (ML-based) IDS for IoT system in Table I. We present the ML mechanisms, datasets for evaluation, and security threats that IDS targets. We categorize the detection mechanisms into four different types:

- *Type 1* mechanisms learn the pattern of the benign traffic and reject network traffic that deviates from the learned pattern. Only benign traffic is used for learning.
- *Type 2* mechanisms learn a binary classifier between the benign traffic and intrusion traffic. Both benign data and intrusion data are used for learning.
- *Type 3* mechanisms measures the distance (or local density) of a data point to the rest of the data (or the neighbors) to detect potential outliers.
- *Type 4* mechanisms learn the pattern of the benign traffic. During the pattern learning, it try to extract the key features that make normal pattern as distinguishable from the intrusion traffic data as possible. This type of detection then detects intrusions by measuring network traffic's deviation from the normal pattern. Both benign data and intrusion data are used for learning.

We have summarized both traditional ML methods (e.g., naive Bayes, decision tree, and random forest) and deep neural networks (e.g., gated recurrent unit, autoencoder, and convolutional neural network) for IDS. As shown in the table, the evaluation datasets include both self-generated data using IoT devices or emulation platforms and well-published datasets. We hope the list of the published datasets will encourage more research on IDS. Due to space limitations, not all the reviewed related papers

TABLE I
REVIEW OF MACHINE-LEARNING-BASED (ML-BASED) INTRUSION DETECTION SYSTEM FOR IoT SYSTEM

Paper	ML Mechanism	Detection Type	Dataset	Security Threats
Kitsune [24], 2018	an ensemble of Autoencoder	Type 1	self generated data on 9 IoT devices	Recon., Man-in-the-Middle, DoS, and Mirai Botnet.
EMFC [25], 2018	ELM-based semi supervised Fuzzy C-Means	others	NSL-KDD [23]	DoS, Probing, U2R, R2L
DIoT [5], 2019	Gated Recurrent Units	Type 1	self-generated data on 33 IoT devices	Mirai Botnet
ID-CVAE [26], 2017, ICVAE [27], 2019	conditional variational autoencoder	Type 1	NSL-KDD [23], UNSW-NB15 [28]	Exploits, Fuzzers, Recon., Analysis, Backdoor, Shellcode, Worms, etc.
TDTC [29], 2019	Naive Bayes and K Nearest Neighbor	Type 2, 3	NSL-KDD [23]	DoS, Probing, U2R, R2L
EIDS [30], 2019	Ensemble of Decision tree, Naive Bayes, and ANN	Type 1	UNSW-NB15 [28], NIMS [31]	Analysis, Backdoor, DoS, Exploit, Fuzzers, Generic, Recon., etc
AE-DNN [32], 2019	Autoencoder plus deep neural network (DNN)	Type 2	AWID dataset [33]	Flooding, impersonation, and injection
RF-SD-IoT [34], 2020	Random Forest	Type 2	self-generated data using Mininet-WiFi emulator [35]	SYN, ping flood, UDP port scan and UDP flood
IoTDefender [36], 2020	Convolutional neural network	Type 2	NSL-KDD [23], CICIDS2017 [37], Datasets [38] [24]	PortScan, DDoS, ARP Spoofing, ARP MitM, Fuzzing, Mirai, etc.
SDRK [39], 2020	deep neural network with random sampling-K-means	Type 2	NSL-KDD [23]	DoS, Probing, U2R, R2L
Passban [14], 2020	Isolation Forest, Local outlier factor	Type 3	self generated data on 6 IoT devices	Port Scan, HTTP&SSH Brute Force, SYN flood
SENTINEL [40], 2021	Naive Bayes, Decision Tree, Logistic Regression, and Random Forest Classifier	Type 2	Self-generated data from 9 IoT devices	Network scan, Exfiltration, C&C keep alive, Black/grey hole
Proposed	Contrastive-learning-based anomaly detection	Type 4	BaIoT dataset [41] and NSL-KDD [23]	DoS, Probing, U2R, R2L, Marai, and BashLite

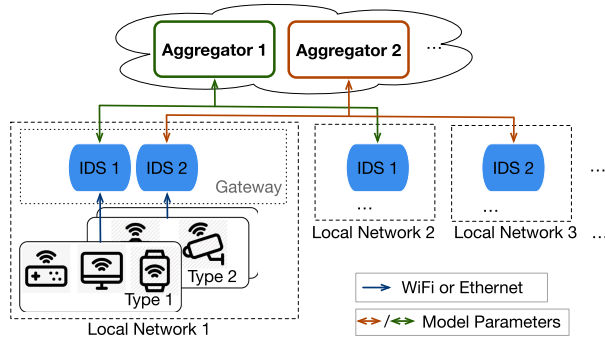


Fig. 1. The system design of our device-type-specific IDS-FeCo. Local gateways cooperatively train an IDS model for each type of IoT devices through FL. The detail of the IDS shown as a blue icon in this figure is depicted in Fig. 3.

are shown in the table. Please refer to survey papers [49], [50] to get connected to ML-based IDSs.

III. SYSTEM MODEL AND THREAT MODEL

We assume an IoT network with multiple types of IoT devices (e.g., IP camera, smart light) connected via a local gateway. As shown in Fig. 1, we focus on the device-type-specific IDS design. For each type of device, a number of gateways cooperatively learn an IDS model through an FL framework. An FL system involves two main components: local gateways (i.e., clients) and a model aggregator, which we describe below.

A **local gateway** $\mathcal{G}$ is a client of the FL system and manages IDSs for detecting compromised IoT devices in the local network. $\mathcal{G}$ may manage multiple IDSs if there are multiple device types present in the local network. $\mathcal{G}$ can choose whether or not to participate in the learning process based on its computation

capability, which results in two operation modes: learning mode and consumer mode. $\mathcal{G}$ in learning mode is a client of the FL system, while $\mathcal{G}$ in consumer mode only request an IDS model from a central server. In the rest of the paper, the default mode for $\mathcal{G}$ is learning mode. $\mathcal{G}$ is also responsible for identifying the type of devices when a new IoT device joins the local network. Such device-type-identification techniques were well studied in [51], [52], [53]. We assume $\mathcal{G}$ to have access to the network traffic of the IoT devices in the local network, which is the same as [5], [24].

A **model aggregator/Server** $\mathcal{S}$ is responsible for aggregating the parameter updates of an IDS model from $\mathcal{G}$ s and sending the up-to-date IDS model to $\mathcal{G}$ s. Typically, $\mathcal{S}$ would be run by a service provider such as Amazon, Google, Microsoft, etc.

A single aggregator $\mathcal{S}$ is responsible for training an IDS model for one type of IoT device. The learning process between aggregator $\mathcal{S}$ and $\mathcal{G}$ can be described as follows. Initially, $\mathcal{S}$ randomly initializes a global model Θ . An FL iteration starts with participant selection at $\mathcal{S}$. $\mathcal{S}$ chooses a subset of local gateways $\mathcal{G}$ s that maintain an IDS model for the targeted type of IoT device and distributes the current global model Θ to the selected $\mathcal{G}$ s to initialize their local IDS models. Each selected local gateway $\mathcal{G}$ proceeds to train its IDS model using data generated by local IoT devices. After several local training epochs, the local gateways $\mathcal{G}$ s upload their model weights θ_i to $\mathcal{S}$. $\mathcal{S}$ updates the global model Θ using the received model weights according to an aggregation rule such as *FedSGD* or *FedAvg* [11], completing one FL iteration. After multiple FL iterations, $\mathcal{S}$ can obtain a well-generalized model that can be used for intrusion detection at local $\mathcal{G}$ s.

Threat Model: IoT devices are easy targets of many network-based intrusions, such as unauthorized access, address spoofing,

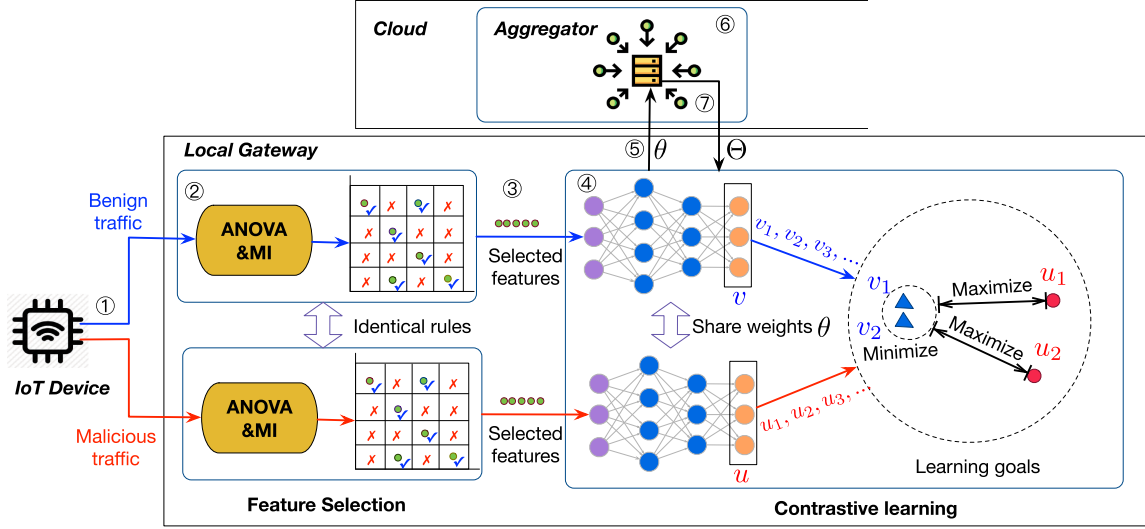


Fig. 2. The workflow of one learning iteration of FeCo. The core of FeCo (i.e., Step 4) is further illustrated in Fig. 3.

TABLE II
CLASSES OF INTRUSIONS AND SUB-CLASSES IN THE NSL-KDD DATASET

	DoS (10)	Probe (6)	R2L (16)	U2R (7)
Attacks in both Training & Testing Set	back, land, Neptune, pod, smurf, teardrop	ipsweep, nmap, portsweep, satan	ftp_write, guesspasswd, imap, multihop, phf, warezmaster	bufferoverflow, loadmodule, perl, rootkit
Attacks only in Testing Set	apache2, mailbomb, processtable, udpstorm	mscan, saint	httptunnel, named, sendmail, snmpgetattack, snmpguess, xlock, xsnoop, worm	ps, sqlattack, xterm
Attacks only in Training Set			spy, warezclient	

false data injection, disruption of network connectivity. Once compromised, they tend to be exploited to launch attacks to other network components or services. In this paper, our goal is to detect malicious network traffic for either purposes.

IV. FECO DESIGN

A. Workflow of IDS

The workflow of the proposed FeCo is illustrated in Fig. 2, which offers an overview of its operations. When joining the learning system, a local gateway $\mathcal{G}$ requests initial IDS model parameters based on the types of IoT devices and initializes its IDS model with the received parameters. During a learning iteration, $\mathcal{G}$ (1) collects raw network traffic data and (2) extracts *significant features* from the raw data. Next, (3) the extracted features are fed into the IDS model. $\mathcal{G}$ then (4) trains the local IDS model using the contrastive learning algorithm and the preprocessed data, and (5) uploads the model parameter update to model aggregator $\mathcal{S}$. $\mathcal{S}$ (6) aggregates received model parameter updates from multiple local gateways and uses them to update the global model. (7) The updated global model is distributed to local gateways for another learning iteration. Steps 1–7 repeat until the global model converges. We can see from Fig. 2 that there are three important components in FeCo including *Feature Selection*, *Contrastive Learning*, and *Federated Aggregation*. We will introduce the three components in detail respectively in the rest of this section. The symbols used in the description and their meanings are listed in Table III.

TABLE III
SYMBOL DEFINITION

Symbol	Definition
x_i	feature vector of a traffic flow, and $x_i \in \mathbb{R}^d$
y_i	label, $y_i \in \{0, 1\}$ where 0 for benign and 1 for intrusion
z_i	output feature representation of x_i , and $z_i \in \mathbb{R}^o$
θ	model parameter
f_θ	mapping function from x to z , $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^o$
$\mathbf{v}_i$	output feature representation of x_i if $y_i = 0$
$\mathbf{u}_i$	output feature representation of x_i if $y_i = 1$
N, M	benign records number and intrusion records number
$\bar{z}$	normal template, i.e., average of normalized z s
$S(x_i)$	the cosine similarity of x_i with template $\bar{z}$
p	percentage value, $0 \leq p \leq 100$
ρ	threshold for determining normal or intrusion

B. Feature Selection

In FeCo, We choose to perform feature selection to remove features that are explicitly demonstrated irrelevant to intrusion detection. We propose a two-step feature selection scheme to select essential features for IDS. By removing redundant features, we simplify the model architecture and reduce the training time as well.

1) *Redundant Feature Removal*: Not all the statistical attributes contain unique information of an individual traffic flow. For example, a feature with zero variance exhibits a constant value in the dataset, and thus it contains no useful information for intrusion detection. Therefore, we first remove the zero-variance features. Furthermore, a feature vector may contain redundant information if there exist multiple highly correlated features. If

we use all the highly correlated features for model training, it is likely to cause overfitting because there exists an implicit emphasis on these correlated features. To reduce the risk of overfitting, we use the Spearman rank correlation coefficient [54] to quantify the correlations among the numerical features and then keep only one feature for each set of highly correlated features.

2) *Feature Importance Ranking*: In FeCo, we propose *feature importance ranking* to rank the importance of features. We employ different methods to measure the importance of categorical features (e.g., the protocol type and service type) and numerical features (e.g., source bytes) since they contribute to the final prediction differently. Specifically, we utilize mutual information (MI) between a feature and the output label to measure the importance of a categorical feature. A zero MI implies that the output label is independent of the target feature. The MI ranges from zero to one, and a higher MI means a higher significance. For numerical features, we employ analysis of variance (ANOVA) to evaluate feature importance. In ANOVA, the observed values of a feature are divided into two groups that are attributable to different values of the label. ANOVA measures whether or not the target feature is statistically different in these two groups. In practice, the ANOVA score is the ratio of the variance between the two groups to the variance within the same group. A larger ANOVA score means higher importance.

The analysis of the proposed feature selection methods and the impact of such data preprocessing on the performance of FeCo are evaluated and reported in Section V-C.

C. Contrastive-Learning-Based IDS

Contrastive-learning-based IDS is the building block of FeCo. It is deployed for the training process at each $\mathcal{G}$. Contrastive learning is first proposed to improve recognition accuracy in the computer vision field. Our goal of deploying contrastive learning is to train a model that produces similar representations for all normal traffic instances and make the intrusion representations far from normal representations. In the following, we first introduce the basics of contrastive learning. Then we depict the proposed contrastive-learning-based binary IDS.

1) *Basics of Contrastive Learning*: Conventional contrastive learning is a type of self-supervised machine learning that acquires feature representations for datasets without requiring labeled data. In essence, contrastive learning trains a model to recognize similarities and differences in data by guiding it to generate similar representations for similar data points and distinct representations for different data points. This learning approach allows model to extract high-level representations from raw data, which can then be utilized to various downstream tasks, such as classification and segmentation. Recently, contrastive learning has seen significant success, as evidenced by [55], [56], [57]. We use the popular SimCLR [55] as an example to introduce the concept formally.

SimCLR learns representations by enhancing the agreement between differently augmented views of the same example, using a contrastive loss in the latent space. The learning framework consists of four steps: 1) Data Augmentation: Create two views

of the same data point, denoted as $\tilde{x}_i$ and $\tilde{x}_j$. These two generated views form a positive pair. 2) Base Encoder: Utilize a neural network f as the base encoder to transform the augmented data into new representation vectors: $h_i = f(\tilde{x}_i)$, and $h_i \in \mathbb{R}^d$ where d denotes the dimension of new representation vectors. 3) Projection Head: Employ a small neural network, namely a projection head g , to map the extracted representation h_i to a new space z where the contrastive loss is calculated. The mapping from h_i to z_i can be represented as $z_i = g(h_i)$. 4) Contrastive Learning Objective: The goal is to learn a similar representation for each positive pair. Assuming there are N positive pairs (i.e., $2N$ data points), the contrastive loss for any positive pair $\tilde{x}_i, \tilde{x}_j$ can be defined as:

$$l_{ij} = \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j) / \tau)}{\sum_{k=1}^{2N} \mathbb{1}[k \neq i] \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k) / \tau)}, \quad (1)$$

where $\text{sim}(\mathbf{z}_i, \mathbf{z}_j) = \frac{\mathbf{z}_i^T \mathbf{z}_j}{\|\mathbf{z}_i\| \|\mathbf{z}_j\|}$ denotes the cosine similarity, $\mathbb{1}[\cdot]$ represents the indicator function, and τ is a hyperparameter. The final loss is computed across all positive pairs. SimCLR iteratively minimizes the loss to learn an encoder f that can produce nearly identical representation vectors for two augmented views originating from the same data points. Our algorithm extends the idea of SimCLR by leveraging labeled data to define a new loss function.

2) *Proposed Algorithm*: We assume each record in the training dataset consists of two fields: input feature vector $x_i \in \mathbb{R}^d$ and output label $y_i \in \{0, 1\}$ where 0 indicates a normal traffic flow and 1 indicates an intrusion. The goal of the contrastive learning algorithm is to learn a new representation (rather than a label) for each input instance. Specifically, contrastive learning trains an ANN model that takes $x_i \in \mathbb{R}^d$ as input and outputs a new representation $z_i \in \mathbb{R}^o$. The ANN model can be represented by a function $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^o$ where θ denotes the model parameters. The ANN model of FeCo consists of four layers: the input layer, two hidden layers, and finally the output layer. The corresponding size of each layer is d , 128, 256, and o , respectively. The default value for o is 128.

For convenience, we use $\mathbf{v}_i = f_\theta(x_i)$ to denote the output of a benign input x_i and $\mathbf{u}_i = f_\theta(x_i)$ to denote the output of an intrusion input x_i . We assume the dataset contains N normal traffic flows and M intrusion traffic flows. For each pair of normal inputs, we can obtain representation $\mathbf{v}_i$ and representation $\mathbf{v}_j$. Our goal is to maximize the similarity between $\mathbf{v}_i$ and $\mathbf{v}_j$ and minimize the similarity between $\mathbf{v}_i$ and $\mathbf{u}_m$ ($m \in [M]$) (we define $[M] := \{1, 2, \dots, M\}$). The likelihood that $\mathbf{v}_i$ is closest to $\mathbf{v}_j$ compared to $\mathbf{u}_m$ is represented by

$$l_{ij} = \frac{\exp(\mathbf{v}_i^T \mathbf{v}_j / \tau)}{\exp(\mathbf{v}_i^T \mathbf{v}_j / \tau) + \sum_{m=1}^M \exp(\mathbf{v}_i^T \mathbf{u}_m / \tau)}, \quad (2)$$

where $0 < \tau < 1$ denotes the temperature coefficient, and such τ makes the likelihood distribution more peaked and narrow compared to the version without temperature coefficient (i.e., $\tau = 1$). It helps the algorithm to capture the similarity more efficiently, and it is a hyper-parameter to be tuned. We then define a loss function $\mathcal{L}_{ij}$ as the negative logarithm of the likelihood

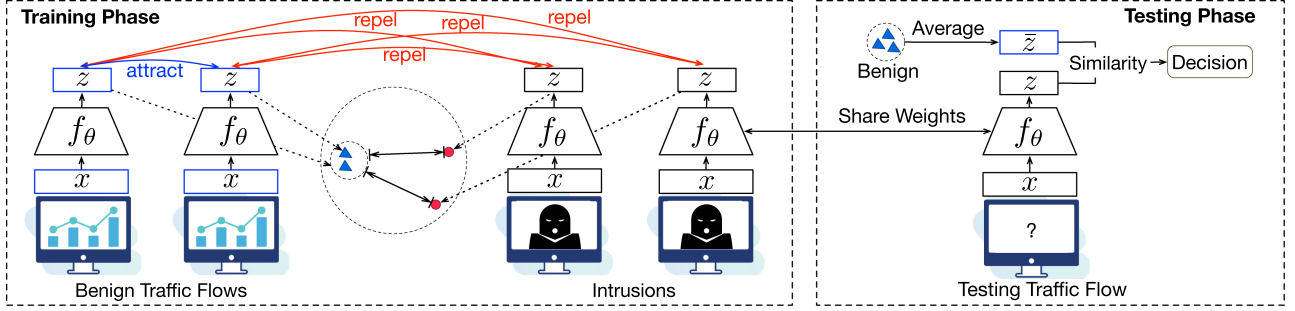


Fig. 3. The training process and testing process of the contrastive-learning-based IDS.

function.

$$\mathcal{L}_{ij} = -\log(l_{ij}) \quad (3)$$

Similarly, we can obtain $\mathcal{L}_{ji}$. The overall loss function of the pair $\mathbf{v}_i$ and $\mathbf{v}_j$ is the summation of $\mathcal{L}_{ij}$ and $\mathcal{L}_{ji}$. We then sum up the loss functions for all pairs of normal vectors and obtain a loss function $\mathcal{L}$:

$$\mathcal{L} = \frac{1}{N(N-1)} \sum_{i=1}^N \sum_{j=i+1}^N \mathcal{L}_{ij} + \mathcal{L}_{ji}. \quad (4)$$

We use a stochastic gradient descent (SGD) optimizer to minimize the loss function $\mathcal{L}$. By minimizing the loss function, we can achieve the goal of maximizing the similarity among normal representations and minimizing the similarity between normal representations and intrusion representations.

We intuitively interpret the learning process in Fig. 3 as well. We can see that the normal z s (i.e., $\mathbf{v}_i$) attracts each other while they repel intrusion z s (i.e., $\mathbf{u}_j$) in the training process. After several iterations, we can learn a model f_θ that outputs a new representation for each input instance. In the new feature space, the representations of benign traffic flows are expected to fall into a compact cluster, while those of intrusion traffic flows are far from such a cluster. In the testing phase, we collect the representations of benign traffic flows and use the average value of the normalized representations as the normal template $\bar{z}$:

$$\bar{z} = \frac{1}{\sum_i \mathbb{1}(y_i = 0)} \sum_i \left(\mathbb{1}(y_i = 0) \frac{f_\theta(x_i)}{\|f_\theta(x_i)\|_2} \right), \quad (5)$$

where $\mathbb{1}(\cdot)$ is the indicator function, and $\mathbb{1}(y_i = 0)$ equals 1 if $y_i = 0$, $\|\cdot\|_2$ denotes the L^2 -norm function and $\frac{f_\theta(x_i)}{\|f_\theta(x_i)\|_2}$ represents a normalized representation. After obtaining the normal template $\bar{z}$, we utilize the cosine similarity estimator to measure the similarity $S(x_j^{test})$ between an upcoming traffic flow x_j^{test} and the normal template:

$$S(x_j^{test}) = \frac{\bar{z}^T f_\theta(x_j^{test})}{\|\bar{z}\| \times \|f_\theta(x_j^{test})\|}. \quad (6)$$

The similarity score $S(x_j^{test})$ ranges from 0 to 1. We need a threshold score $0 \leq \rho \leq 1$ for determining whether x_j^{test} is an anomaly or not. In our paper, we obtain the threshold ρ by calculating the statistics of the scores of benign training data. Particularly, we first sort the scores of benign data in ascending order and obtain the sorted sequence $\mathbf{S} = [S_1, S_2, \dots, S_N]$. Then

we select the p -th percentile of $\mathbf{S}$ as the threshold ρ . In FeCo, we first calculated the index r of the score to select:

$$r = \left\lfloor \frac{p}{100} * N \right\rfloor, \quad (7)$$

where $\lfloor \cdot \rfloor$ denotes the round down function, and $0 \leq p \leq 100$ represents a percentage number. We obtain $\rho = S_r$ which is the r -th entry of $\mathbf{S}$. We can manually select a value for p . We should select a small value for p (e.g., $p = 5$) as a larger p leads to a higher FPR. The final decision $\hat{y}_j$ is made by

$$\hat{y}_j = \mathbb{1}(S(x_j^{test}) < \rho). \quad (8)$$

An input instance is predicted as an intrusion if its similarity score is smaller than threshold ρ .

D. Federated Aggregation

We build FeCo by incorporating the contrastive-learning-based IDS into the federated learning framework. In that case, each client participates in the FL process by providing its model parameter update. We utilize the FedAVG [11] algorithm to aggregate the updates from multiple clients. In time step t , the model aggregator $\mathcal{S}$ computes the global model parameter Θ_t by:

$$\Theta_t = \Theta_{t-1} + \sum_i c_i * (\theta_i - \Theta_{t-1}), \quad (9)$$

where θ_i is the local model parameters at client i and c_i is a weight coefficient. In our paper, c_i based on the size of local training dataset at client i . Particularly, we define c_i as the ratio of the size of the local training dataset at client i to the number of total training samples at all selected clients.

V. EXPERIMENTAL RESULTS

A. Datasets and Experiment Settings

We implement FeCo in the PyTorch platform [58]. We ran all the experiments on a server equipped with an Intel Core i7-8700 K CPU 3.70 GHz $\times$ 12, a GeForce RTX 2080 Ti GPU, and Ubuntu 18.04.3 LTS. We experiment with FeCo using two network traffic datasets including the NSL-KDD dataset and BaIoT dataset. The NSL-KDD dataset is a network traffic dataset that is widely used for IoT scenarios [18], [25], [49]. We evaluate FeCo on the NSL-KDD dataset in order to provide comparisons

with other IDSs [18], [25], [49]. The BaIoT dataset that is dedicated to IoT devices.

The NSL-KDD dataset [23] includes benign traffic and four categories of intrusions, i.e., DoS, Probing, Remote-to-Local (R2L), and User-to-Root (U2R). Each category of intrusion contains several sub-classes as shown in Table II. The whole dataset includes a training set and a testing set, and the training set contains 125,973 records while the test set 22,544 records. Note that some intrusion sub-classes exist only in the testing set, i.e., they are unseen in the training set (e.g., mscan, sqlattack), which makes it possible to evaluate FeCo against zero-day attacks. Each record consists of 41 attributes extracted from a traffic flow and a label indicating its category (i.e., Normal, DoS, Probing, R2L, or U2R). In practice, it is easy to extract attributes from network packets by using existing packet analyzers (e.g., WireShark³).

The BaIoT dataset [41] contains the traffic data gathered from 9 commercial IoT devices authentically infected by Mirai and BASHLITE. Benign traffic collected at normal running phase is also included. Both Mirai and BASHLITE is composed of five sub-class attacks: ‘scan’, ‘ack’, ‘syn’, ‘udp’ and ‘udpplain’ for Mirai; and ‘scan’, ‘junk’, ‘udp’, ‘tcp’, and ‘combo’ for BASHLITE. The whole dataset contains 6,506,674 malicious records and 556,932 benign records, and each record contains 115 attributes. The dataset is available from <https://archive.ics.uci.edu/ml/machine-learning-databases/00442/>

Some papers [59], [60] reported detection accuracy as high as 99% on the NSL-KDD dataset. However, these works introduce a new splitting on the dataset. They either combine the training set and testing set into one then randomly split it into two sets for training and testing, or directly split the training set into two sets. Such arrangements can only demonstrate the effectiveness of IDS models in detecting known intrusions but not unseen intrusions. In this paper, we train our IDS model with the training set and evaluate it using the testing set to explicitly show its performance in detecting unseen intrusions.

Other default settings of FeCo are shown as follows. In order to better simulate the distributed characteristics of a real FL system, we choose 50 clients which is relatively larger than 10 and 15 used in [5], [42]. We generate data for clients by splitting the whole dataset. Therefore, the number of clients and the size of local data would be inversely proportional to each other. By using different splitting strategies, we obtain both IID data and non-IID data (See Section V-D2 for detail). For data preprocessing, we first use our feature selection scheme to remove 10 attributes from the 41 attributes. Then we use the one-hot encoding method to map the remaining 31 attributes into 112 input features. The size of the input layer and the output layer of the ANN model in FeCo are $d=112$ and $o=128$, respectively. The machine learning baselines used for comparison are imported from scikit-learn [61]. In FeCo, each client uses an SGD optimizer with a learning rate of 0.001 for local training. The clients perform four epochs of local training before sending its model parameters to $\mathcal{S}$ in each FL round. We set the number of total FL rounds as 15.

B. Evaluation Metrics

For a binary detection problem, there are four important terms: True Positive (TP) means correctly detected as an intrusion; False Positive (FP) means incorrectly detected as an intrusion; True Negative (TN) means correctly detected as benign; False Negative (FN) means incorrectly detected as benign. We use N_* to represent the number of $* \in \{TP, TN, FP, FN\}$. We compute five evaluation metrics: accuracy $A = \frac{N_{TP} + N_{TN}}{N_{TP} + N_{FP} + N_{TN} + N_{FN}}$, recall $R = \frac{N_{TP}}{N_{TP} + N_{FN}}$, precision $P = \frac{N_{TP}}{N_{TP} + N_{FP}}$, F1 score $F = \frac{2 \times P \times R}{P + R}$, and False Positive Rate (FPR) $fpr = \frac{N_{FP}}{N_{FP} + N_{TN}}$. We also provide the receiver operating characteristics (ROC) curve by plotting R against FPR at various threshold settings. The AUC score is defined as the area under the ROC curve.

C. Feature Selection

Here we show the process of our two-step feature selection scheme using NSL-KDD as an example.

We first remove the zero-variance feature (i.e., ‘num_outbound_cmds’) from the dataset. Then we evaluate the feature correlation and show the heat map of the correlation matrix in Fig. 4(a). We further perform hierarchy clustering on the computed correlations among features, shown in Fig. 4(b). Focusing on the height at which any two objects are joined together, we can see the height of the link that joins ‘feature 0’ and ‘feature 23’ is the smallest, indicating that the two features are the most correlated. We can further obtain the second most correlated feature pair and the third most correlated feature pair. We randomly remove one feature from the three feature pairs as removing these features does not degrade the detection performance.

Second, we evaluate importance scores for all features. We show the ranking of MI scores and ANOVA scores in Fig. 4(c). The MI score of Feature 32, 33, 36 (i.e., ‘land’, ‘root_shell’, ‘is_host_login’) is zero, implying that they are irrelevant to intrusion detection. Therefore, we remove the three features. Note that the ANOVA scores are shown on a logarithmic scale as the scores vary dramatically among different features. Unlike MI scores, ANOVA scores have no zero entries. We start with removing the features with the lowest ANOVA score. We vary the number K of features to remove. Intuitively, useful information may be discarded if K is too large, while redundant information may result in overfitting if K is too small. We tune $K \in \{0, 1, 2, 3, 4\}$ to show the impact of K on detection accuracy. Fig. 5(a) shows the performance of FeCo with different K . We can see that both accuracy and the F1 score increase with $K \leq 3$, implying that removing features with low significance could boost the detection performance. We select $K = 3$ as the accuracy peaks at $K = 3$.

D. FeCo Performance on NSL-KDD

1) *Centralized Setting*: We first investigate the performance of FeCo under the centralized setting to focus on evaluating our contrastive-learning-based IDS. We evaluate the impacts of global aggregation of FL in next part. We compare the performance of FeCo to both state-of-the-art IDSs [18], [25], [29] and

³<https://www.wireshark.org/>

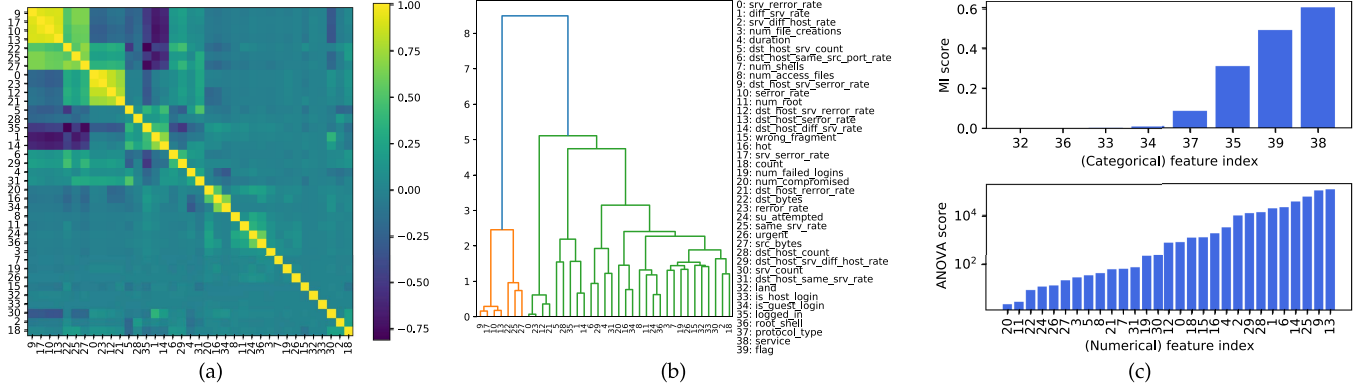


Fig. 4. (a) The heat map of the Spearman correlations. (b) The dendrogram of the correlation clustering. (c) The ranking of MI scores and ANOVA scores for categorical features and numerical features, respectively.

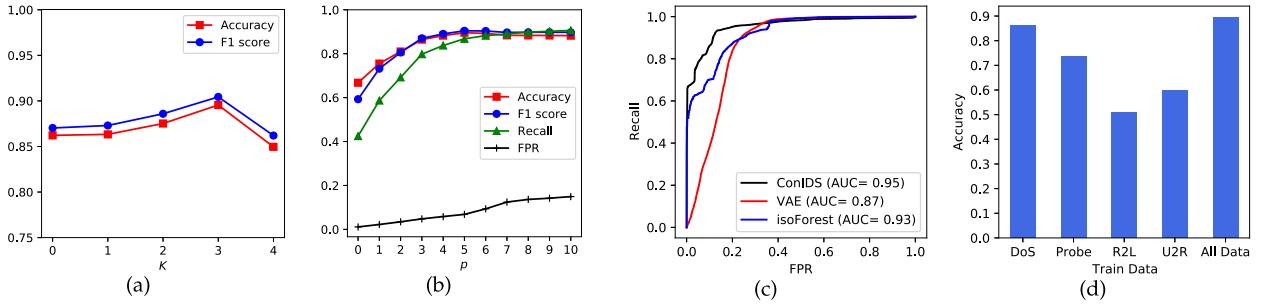


Fig. 5. FeCo performance under the centralized setting. (a) The accuracy and F1 scores of FeCo with different number of removed features K . (b) Accuracy, F1 score, recall, and FPR of FeCo with different percentage value p . (c) ROC curves of FeCo, VAE, and IsoForest. (d) Accuracy of FeCo with training data containing only one attack (DoS, Probe, R2L, or U2R).

some other widely-used machine learning baselines including support vector machine (SVM), variational autoencoder (VAE), isolation forest (IsoForest), multilayer perceptron (MLP), logistic regression (LGR), Bernoulli naive Bayes (BNB), K-nearest neighbors (KNN), and decision tree classifier (DTC).

As shown in Section IV-C, the threshold for intrusion detection is computed by manually selecting the quantile number p . Fig. 5(b) shows the performance of FeCo with different p value. We can see that the recall increases with p and the FPR also increases as expected. The accuracy and F1 score first increase dramatically with the increase of p then decrease slowly. Given such results, it is intuitive that one can select the value for p based on the desired FPR. We should select a small p if we desire a low FPR. Ideally, FPR should be very close to the value of $p/100$ if the learned model generalized well on the testing set. In practice, we set $p = 5$, and get $FPR = 6.8\%$ in the testing set. This discrepancy may be due to novel attacks in the testing set. However, we believe that the small gap between $FPR = 6.8\%$ and $5/100$ confirms that FPR would approximate the value of $p/100$.

We show the detection performance of FeCo and other baselines in Table IV. We set $p = 5$ to obtain these results. FeCo achieves detection accuracy as high as 89.55%. From Table IV we can see that FeCo outperforms other methods by achieving both the highest recall and the highest accuracy. Note that some entries of Table IV are missing because they were not provided in the references. To demonstrate FeCo's capability in detecting

TABLE IV
PERFORMANCE (%) OF FECO IN CENTRALIZED SETTING

Method	Accuracy	Recall	Precision	F1 score	FPR
FeCo (Proposed)	89.55	86.80	94.39	90.44	6.82
ESFCM [25]	80.69	80.72	80.85	80.45	-
Two-Tier [29]	-	82.00	-	-	5.43
TDTC [18]	-	84.86	-	-	4.86
VAE	81.77	73.71	92.78	82.15	7.58
IsoForest	79.54	69.35	92.90	79.42	7.00
MLP	79.65	66.41	96.85	78.80	2.85
SVM	77.26	64.65	93.35	76.40	6.09
LGR	75.83	66.05	88.59	75.68	11.24
BNB	77.63	63.27	96.12	76.31	3.38
KNN	77.58	62.39	97.23	76.01	2.35
DTC	77.36	64.01	94.44	76.30	4.98

TABLE V
RECALL (%) OF FECO FOR NOVEL TESTING ATTACKS

FeCo	VAE	IsoForest	MLP	SVM	LGR	BNB	KNN	DTC
78.45	70.08	61.94	48.43	35.60	52.24	59.84	42.72	49.73

zero-day attacks, we split the testing attacks into known attacks and novel attacks (i.e., attacks unseen in the training data). We present the recall of FeCo for novel testing attacks in Table V. We can see FeCo achieves a recall 8% to 42% higher than other machine learning baselines. SVM, BNB, KNN, DTC, and MLP achieve less than 50% recall, which indicates they are not suitable for detecting zero-attacks.

TABLE VI
DETECTION PERFORMANCE (%) IN CENTRALIZED SETTING

		Prediction		Total	Recall (%)
		Normal	Intrusion		
Ground truth	Normal	9049	662	9711	-
	DoS	212	7246	7458	97.16
	Probe	95	2326	2421	96.08
	R2L	1352	1402	2754	50.91
	U2R	35	165	200	82.50

Among all the methods shown in Table IV, FeCo, IsoForest, and VAE share a similar detection strategy that detects intrusions by learning a model to characterized benign traffic. These methods compute a score for inputs using the learned model. The output labels are predicted by comparing the scores to a threshold. Consequently, a different threshold would lead to different detection accuracy. We plot the ROC curves of the three methods in Fig. 5(c). Each point in one curve corresponds to the recall and FPR under one specific threshold. We can see that FeCo achieves the highest recall under the same FPR compare to IsoForest and VAE. Obviously, FeCo outperforms the other two methods in terms of AUC score as well.

To better understand the performance of FeCo, we further look into the recall of FeCo for each category of intrusion attacks (i.e., DoS, Probing, R2L, and U2R). Intuitively, for one attack category, the recall means the proportion of records detected as intrusions, i.e., labelled with ‘1’. Formally, the recall of each attack category $i \in \{1, 2, 3, 4\}$ (1 for ‘DoS’, 2 for ‘Probe’, 3 for ‘U2R’, and 4 for ‘R2L’) is defined as

$$R(i) = \frac{\sum_j \mathbb{1}(\hat{y}_j = 1 \ \& \ y_j^{multi} = i)}{y_j^{multi} = i}, \quad (10)$$

where $\hat{y}_j$ denotes the prediction of FeCo on the j -th data record, and it takes a value of either 0 or 1 as FeCo is a binary IDS. Note that we have true labels of whether a testing record is ‘Normal’ or ‘Intrusion’ (i.e., $y_j \in \{0, 1\}$) and whether a testing record is ‘Normal’, ‘DoS’, ‘Probe’, ‘U2R’, or ‘R2L’ (i.e., $y_j^{multi} \in \{0, 1, 2, 3, 4\}$).

In Table VI, we show the confusion matrix including the attack-specific recall. Here we omit the recall of ‘Normal’ as normal traffic is not an attack. From Table VI, we can see that FeCo achieves higher recall on both DoS attack and Probe attack, while the detection rate on R2L is as low as 0.51. To the best of our knowledge, all works with evaluations on the NSL-KDD dataset show a low detection performance on R2L as well [18], [29]. There are two possible reasons behind the low detection rate on R2L: 1) the number of training instances belonging to R2L attack is as small as 995, and 2) there are many new attacks in the testing set that do not exist in the training set as shown in Table II.

We added an ablation study to analyze the effectiveness of each module of FeCo, including the feature selection module denoted by **Fea**, the contrastive-based representation learning module denoted by **Rep**, and the similarity-based anomaly detection module denoted by **Detect**. FeCo includes all the three components and is represented by **Fea(✓)-Rep(✓)-Detect(✓)**. We build three baseline learning mechanisms by dropping one

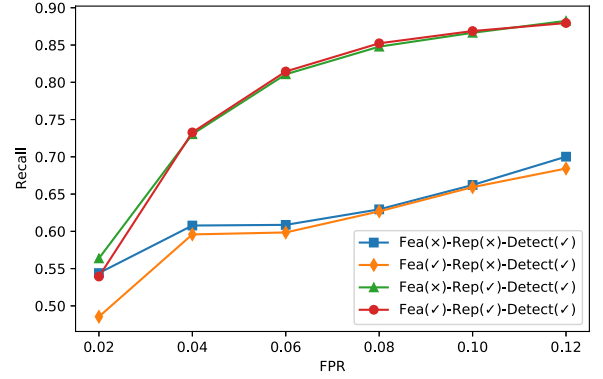


Fig. 6. Ablation study on FeCo to investigate the performance gain obtained by the three major components—**Fea**, **Rep**, and **Detect**.

or two components from FeCo, and denote them as **Fea(×)-Rep(✓)-Detect(✓)**, **Fea(×)-Rep(×)-Detect(✓)**, and **Fea(✓)-Rep(×)-Detect(✓)**. In Fig. 6, we show the recall of FeCo and the other three baselines with respect to various FPRs. The superior recall value of **Fea(×)-Rep(✓)-Detect(✓)** and **Fea(✓)-Rep(✓)-Detect(✓)** compared to the other two baselines indicate that the contrastive-based representation learning contributes the most to the final accuracy. Moreover, the nearly identical recall of **Fea(×)-Rep(✓)-Detect(✓)** and **Fea(✓)-Rep(✓)-Detect(✓)** implies that our feature reduction does not negatively impact the model detection performance.

To investigate the impact of data distribution, we evaluate FeCo under the scenario when we only have partial data drawn from the whole data distribution. In particular, we sample the data belonging to one attack category (e.g., ‘DoS’) and the normal data (i.e., ‘Normal’). In other words, each FeCo model in these experiments is trained from records of *only one* attack category and ‘Normal’ class. As a result, the detection performance of such FeCo models is expected to be lower than that of FeCo models trained from the whole NSL-KDD dataset. We evaluate the detection accuracy with the same testing dataset of NSL-KDD and show the results in Fig. 5(d). We can see the detection accuracy with partial data is lower than that with the whole NSL-KDD dataset, which is as expected. The accuracy of the FeCo model trained with only ‘R2L’ and ‘Normal’ is as low as 50.8%. This observation indicates that self-learning may result in extremely low detection accuracy as the local data may not exhibit the overall data distribution.

2) *Federated Setting*: Here we focus on the performance of FeCo in the federated setting. To present a thorough comparison, we have three different learning frameworks: FL, self-learning, and centralized learning. In self-learning, the training process is done at the local device using the local data. Centralized learning refers to FeCo investigated in Section V-D1. Further more, data distribution is one of the biggest factors that impact FL performance. Therefore, we propose to explore three different data distributions, including one IID data distribution and two non-IID data distributions. For the IID data distribution, we randomly select a subset of each class in the whole training dataset as the local data of one client. In non-IID-1, we assume that each client only has $n \in \{1, 2, 3, 4\}$ out of the total four intrusion

TABLE VII
PERFORMANCE (%) OF FECO IN FL, SELF-LEARNING, AND CENTRALIZED LEARNING FRAMEWORKS

Learning Type	Data Distribution	Accuracy	Recall	Precision	F1 score	FPR	DoS	Probe	R2L	U2R
FL	IID	85.65	81.15	92.73	86.55	8.41	92.69	98.68	35.00	74.00
	non-IID-1	86.23	82.10	92.89	87.16	8.29	93.12	95.67	41.03	72.50
	non-IID-2	85.53	77.90	95.93	85.98	4.26	83.33	95.29	36.49	76.50
Self-learning	IID	81.91	74.53	92.21	82.43	8.33	87.66	92.14	24.33	63.07
	non-IID-1	82.62	75.69	92.39	83.21	8.23	90.48	87.37	25.63	71.81
	non-IID-2	67.66	46.51	93.89	62.21	4.39	54.20	47.22	24.14	59.25
Centralized Learning	-	89.55	86.80	94.39	90.44	6.82	97.16	96.08	50.91	82.50

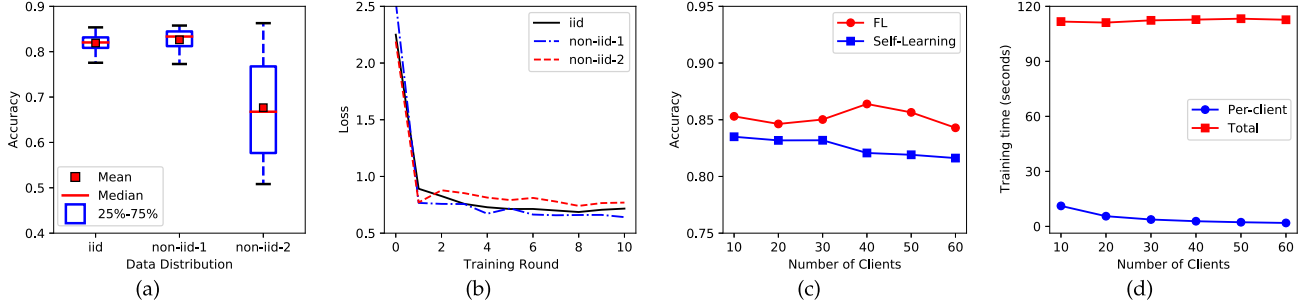


Fig. 7. Evaluations of FeCo. (a) Accuracy distribution of FeCo in self-learning mode. (b) Convergence performance of FeCo in FL mode. (c) Accuracy of FeCo with different number of clients. The size of per-client data get smaller when the number of clients increases since the size of total data is the same; (d) Per-client running time and total running time of all clients.

attack categories. In non-IID-2, each client has $n = 1$ intrusion category. For both non-IID-1 and non-IID-2, we assume all users have benign data, i.e., data from ‘Normal’ class. non-IID-2 is a special case of non-IID-1 and training on non-IID-2 is more challenging than other settings due to fewer attack categories for training. Meanwhile, we use the same testing data in all settings for fair comparison.

We show the detection performance of FeCo with combinations of the three learning types and three data distributions in Table VII. In self-learning, the value of evaluation metrics is shown as an averaged value over all clients. Compared with self-learning, FL achieves higher accuracy, recall, and precision regardless of the data distribution. Furthermore, FL achieves similar detection performance under the three data distributions, implying that FeCo is able to obtain stable learning performance with different data distributions. Unlike FL, the performance of self-learning heavily depends on the data distribution. We can see that self-learning achieves as low as 67.66% average accuracy under non-IID-2 data distribution. One noticeable result is that the detection accuracy of FL is still lower than the centralized learning. In practice, centralized learning is difficult to deploy due to privacy concerns. We further show the box-plot of self-learning accuracy of 50 clients in Fig. 7(a) to further accommodate Table VII (Table VII only shows the average accuracy among clients). We can see that the variance of the accuracy of non-IID-2 is much larger than those of IID and non-IID-1.

We have another observation when comparing the performance of FeCo with different data distributions but the same learning framework. In the FL setting, the accuracy of FeCo on non-IID-1 is 86.23% which is higher than the 85.65% accuracy achieved on IID data. Similar phenomena occur in self-learning as well. The possible reason behind this is that: the IID data

contain a relatively large number of attack sub-classes (i.e., 20) within the small local dataset. Therefore, it becomes difficult to learn a stable representation of benign instances when contrastive learning iteratively pushes benign representations away from so many different attacks. On the contrary, in the non-IID-1 setting, each client possesses only a subset of attack classes thus a smaller number of attack sub-classes. Therefore, it is easier to learn a stable normal template. The performance of non-IID-2 is worse than IID possibly because overfitting occurs as the attack categories in the local data are too limited.

We study the convergence performance of FeCo by plotting the value of loss through the training process. As shown in Fig. 7(b), the loss drops sharply at the beginning of the training process, decreases slowly after certain epochs, and finally stays stable. We can see that the loss of FeCo reaches the stable state fast under the three data distributions, implying that FeCo converges after a small number of learning rounds.

We also explore the scalability of FeCo by varying the number of clients. We show the accuracy of FeCo with different number of clients in Fig. 7(c). We can see that the accuracy of self-learning decreases monotonically with the increase of the number of clients. This is because that the size of the local data also decreases with the increase of the number of clients as they are inversely proportional to each other. However, the accuracy of FL does not show a decreasing trend. This observation indicates that FeCo scales well with the number of clients in the FL mode.

We show the overhead of FeCo in Fig. 7(d). Specifically, we present the per-client running time and the total running time of all clients for one FL round. Note that a FL round means that the selected clients finish training their local models and upload the model parameters for one time. We evaluate the running time with the number of clients ranging from 10 to 60. Note

TABLE VIII
PERFORMANCE OF FeCo ON THE BaIoT DATASET

Index	Devices Make and Model	Device Type	Accuracy (%)	Recall (%)	Precision (%)	F1 (%)	FPR (%)
1	Danmini	Doorbell	99.98	99.99	100.00	99.99	0.16
2	Ennio	Doorbell	99.95	99.99	99.96	99.97	0.77
3	Ecobee	Thermostat	99.98	99.98	100.00	99.99	0.13
4	Phillips B120N/10	Baby monitor	99.94	99.97	99.97	99.87	0.37
5	Provision PT-737E	Security camera	99.96	99.98	99.98	99.98	0.50
6	Provision PT-838	Security camera	99.96	99.98	99.98	99.98	0.34
7	SimpleHome XCS7-1002-WHT	Security camera	99.97	99.98	99.99	99.98	0.37
8	SimpleHome XCS7-1003-WHT	Security camera	99.98	99.98	100.00	99.99	0.31
9	Samsung SNH 1011 N	Webcam	99.95	99.95	99.99	99.97	0.07

that a larger number of clients means a smaller dataset at the local client as discussed above. We can see that the per-client running time decreases with the increase of the number of clients while the total running time stays approximately the same. The per-client running time is as low as 1.878 seconds when the local data contains 2000 records, indicating that FeCo requires relatively little computation resource. We plan to implement FeCo in a gateway device in our future work to demonstrate that FeCo is affordable even in a local gateway with low computation capacity.

E. FeCo Performance on BaIoT Dataset

We further evaluate FeCo using a real-world IoT dataset. The BaIoT dataset includes nine commercial IoT devices as shown in Table VIII. We train a model for each IoT device and evaluate the performance separately.

In order to test the capability of FeCo to detect zero-day attacks, we design our data split method as follows. The BaIoT dataset is composed of ten classes of malicious data: five classes from the Mirai attack family and five classes from the BASHLITE attack family. We randomly select three from the five Mirai attacks as the training data. The remaining two Mirai attacks and all the five BASHLITE attacks are used for testing. There are no Mirai attack records for two of the nine devices (the Ennio doorbell and the SamSung SNH 1011 N webcam). Therefore, we randomly select three from the five BASHLITE attacks as the training data, and the remaining two BASHLITE attacks are used for testing. The benign traffic data is split into 70% for training and 30% for testing.

We show the performance of FeCo on the BaIoT dataset in Table VIII. The BaIoT data is imbalanced: the number of malicious data records (6,506,674) is about ten times of benign data records (556,932). Therefore, we present recall and FPR in addition to detection accuracy to better exhibit the detection performance. FeCo achieves recall as high as 99.99% with a small FPR of 0.16%. Table VIII demonstrates that FeCo is effective in detecting malicious traffic for various types of IoT devices. Furthermore, the high overall recall indicates that the IDS model trained on partial intrusion attacks effectively detects unseen attacks. Other baselines in [41] achieve similar recall but higher FPR. The FPR of Autoencoder, isolation forest, SVM, and FeCo is shown in Fig. 9. We can see IsolationForest is unstable in detecting intrusions among different types of IoT devices as it results in a large FPR on Device 4 and Device 5. FeCo outperforms all the evaluated baselines by achieving a much smaller FPR.

We further explore how FeCo detects intrusions. As discussed in Section IV-C, FeCo computes the cosine similarity of an input with the normal template and flags the input as intrusion if the calculated similarity score is less than a pre-defined threshold. We show the histogram of the similarity scores of test data points in Fig. 8. To give a clear view of the similarity score distribution, we analyze the scores of intrusion records and scores of benign records separately. As shown in Fig. 8(a), two sub-figures are given: the top sub-figure showing the similarity score distribution of the intrusion traffic and the bottom sub-figure showing the similarity score distribution of benign traffic. We can see that similarity scores of intrusion traffic are distributed from -0.56 to 0.88 and has two peaks at around -0.4 and 0.88. The instances that form the left peak and its vicinity represent the Mirai attack, and the instances form the right peak represents the BASHLITE attack. For benign traffic, the similarity score gathers at a very narrow range that is very close to the value of 1.00. The gap between the largest score of intrusion traffic and the smallest value of benign score demonstrates FeCo's capability to detect intrusions. The similarity score distribution of the other eight devices is shown in Fig. 8(b) to (i). Refer to Table VIII for mapping a device number in the caption of Fig. 8 to a real device. We can also see only one peak in the similarity score distribution of the intrusion traffic in Device 2 and Device 9. The observation is because the two devices are only infected by the BASHLITE attack but not the Mirai attack.

VI. COMPUTATION OPTIMIZATION

We further optimize FeCo to make it compatible with computational constrained IoT devices. In this section, we optimize the FeCo by reducing the size of the neural network model used by FeCo. We instrument the optimized FeCo system to a Raspberry Pi to demonstrate its applicability in computation-constrained IoT devices.

A. Model Weights Quantization

In order to reduce the model size, we perform model weights quantization. With a smaller model size, we can achieve the following advantages. A smaller model will occupy less storage space on an IoT device. In this way, an IoT device can save more storage space for its original operation data. Second, a smaller model will use less RAM in the run-time, which saves more memory for other applications of an IoT device. Moreover, quantization will also simplify the calculations involved in the inference, resulting in decreasing the amount of time of a single inference operation (i.e., latency). However,

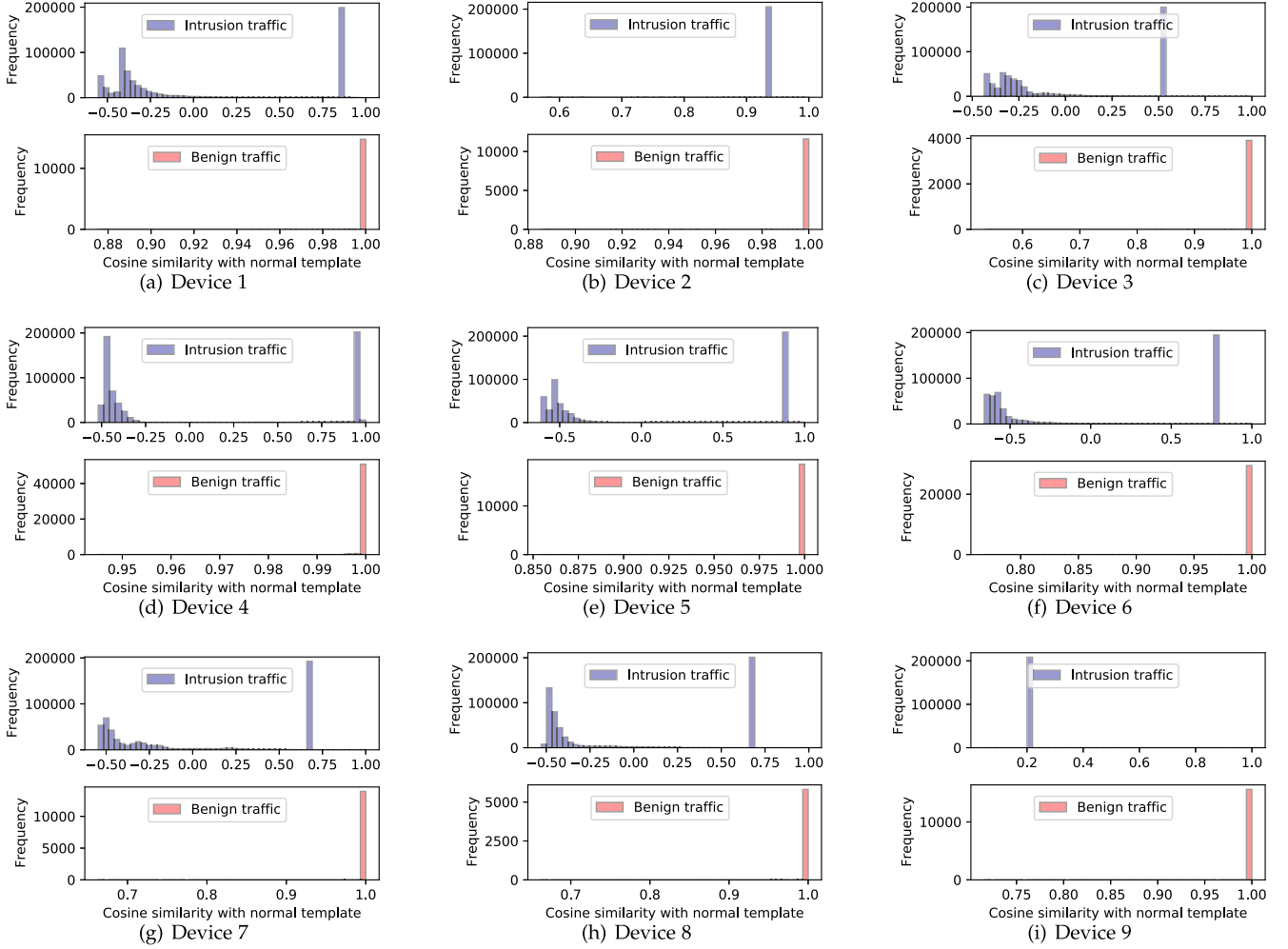


Fig. 8. Cosine similarity of network traffic with the learned template of benign network traffic.

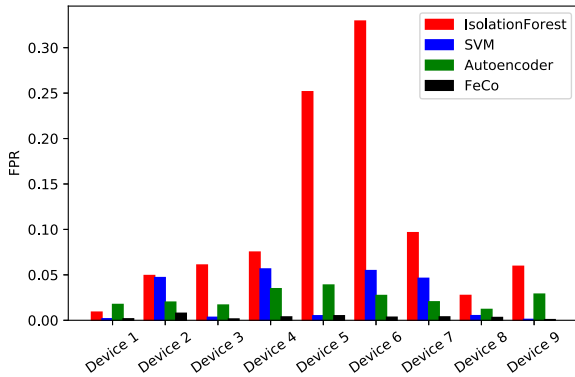


Fig. 9. FPR when the value of Recall is fixed to 0.9998.

model weights quantization also potentially sacrifices the model accuracy. There exists a trade-off between model accuracy and model size. We employ post-training quantization that quantizes the model parameters after a model is trained. There are two weights quantization methods:

- *Int-8 quantization*: The 32-bit float weights of a trained model are quantized to 8-bit integer. The model size will be 4 times smaller.

- *Float-16 quantization*: The 32-bit float weights of a trained model are quantized to 16-bit float. The model size is expected to be 2 times smaller.

We perform model quantization on an open-source framework, i.e., the TensorFlow Lite.⁴ Model quantization in our paper is a post-training process, meaning that the model training stays the same as what is depicted in Section IV. The operational flow is: first, training a model on a local gateway; optimizing the trained model to a smaller size by weights quantization; implementing the optimized model to a Raspberry Pi device. The first two steps can be completed by either a local gateway or a server. We will focus on the third step as we aim to analyze the performance of FeCo when it is implemented on resource-constrained devices (i.e., most IoT devices).

B. Implementation and Evaluation

We prototype our computation-optimized FeCo on a Raspberry Pi 4 equipped with 1.5 GHz quad-core A72 64-bit ARMV8 CPU, 4 GB RAM, 32 GB SIM card memory, and Raspbian OS. This experiment aims to check the feasibility of deploying FeCo

⁴<https://www.tensorflow.org/lite>

TABLE IX

MODEL SIZE, INFERENCE TIME (PER TEST DATA POINT), AND INFERENCE ACCURACY ON ECOBEE THERMOSTAT TESTING DATASET WHEN FeCo IS IMPLEMENTED ON A RASPBERRY PI DEVICE

Quantization method	Model size	Inference time (/record)	Accuracy (%)	Recall (%)	Precision (%)	F1 (%)	FPR (%)
No quantization (Float-32)	254kB	0.152ms	99.96	99.97	100	99.98	0.33
Float-16 quantization	129kB	0.151ms	99.96	99.97	100	99.98	0.33
Int-8 quantization	67kB	0.136ms	99.96	99.97	100	99.98	0.33

on resource-constrained IoT devices. The metrics we evaluate include the testing phase performance and the inference latency of various quantized models.

Table IX demonstrates the performance of FeCo on a Raspberry Pi. In the table, we compare the performance among three model optimization methods: No quantization (Float-32), Float-16 quantization, and Int8 quantization. We witness a significant reduction in the model storage size with model quantization methods. Compared to No quantization, the model storage space is reduced by about half when applying Float-16 quantization. And the storage space is reduced by about 3/4 when applying Int-8 quantization. Further, the inference time is also reduced by applying model quantization. Int-8 quantization saves 0.016 ms for inference on one data record compared to No quantization, which speeds up the inference by 11%. Float-16 quantization has nearly the same inference time as No quantization. This is because the ARM processor does not support 16-bit computation, and the model converted to 16-bit will be upsampled to float 32 before the inference. Most importantly, the Int-8 quantization and Float-16 quantization suffer from no decay in detection performance compared to the Non-quantized model. Therefore, we recommend Int-8 quantization on the CPU-based platform (e.g., the ARM processor). And both Int-8 quantization and Float-16 quantization will be good choices for GPU-based platform as GPU support both 8-bit computation and 16-bit computation. In sum, Table IX demonstrates the optimized model can save storage space and speed up the inference without degrading the detection performance.

VII. DISCUSSIONS

Device type identification is challenging and critical for managing IoT networks [51], [52], [53], and it is also a significant part of FeCo design. The goal of device type identification in this work is to cluster IoT devices together based on their security requirements and vulnerability. It is demonstrated that devices from a given manufacturer running the same version of firmware will have the same vulnerabilities [51]. We borrow their idea to map devices to an abstract *device type* for which the system has learned a specific set of policies. In the proposed FeCo system, we assume IDS service providers (i.e., the IDS learner on the cloud side) will initialize an IDS model in a cloud server only if they identify a type of device on the market. The service provider then shares the policy set with local gateways. Local gateways perform device type identification with the knowledge shared by the cloud server.

System scalability is another crucial issue, and here we discuss the scalability at both local gateways and the cloud server. At the local gateway level, the computation overhead is associated with the number of devices and device types. The number of IDS models maintained by a local gateway will increase linearly with

the number of device types. Within each IDS model, the gateway can manage the training overhead by adjusting the amount of data used for training based on its computation capacity. Additionally, inference computation should not pose a significant problem, as it is generally lighter than training computation. Regarding the cloud server, the communication and computation overhead is determined by the number of participating local gateways. In accordance with federated learning (FL) design principles, the cloud server can control the number of local gateways involved in each learning iteration. Consequently, even if a large number of local gateways volunteer to participate in FL, the cloud server can still select a small portion of them in each learning iteration, ensuring the task remains within its communication and computation capabilities.

VIII. CONCLUSION

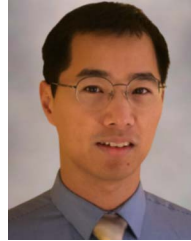
In this paper, we propose FeCo, a machine-learning-based IDS for IoT networks. FeCo incorporates contrastive learning into the federated learning framework to support distributed intrusion detection while preserving user data privacy. More importantly, FeCo features a novel detection method based on network traffic representation learning through contrastive learning. While learning for the network traffic representation, FeCo tries to maximize the distance between benign and malicious samples and minimize the distance among benign samples. This effectively enables FeCo to achieve better detection accuracy than other baselines as FeCo obtains a more stable normal profile of network traffic. In order to avoid overfitting, we further propose a two-step feature selection scheme to remove redundant features before learning. The feature selection scheme also decreases computation complexity, making FeCo more suitable for resource-constrained IoT devices. We demonstrate the high effectiveness of contrastive learning in IDS through extensive evaluations with the NSL-KDD dataset and BaIoT dataset. We perform model optimization by leveraging the model weight quantization method. We implement and evaluate the optimized FeCo model for a low-end device (i.e., Raspberry Pi) to demonstrate the efficiency and effectiveness of FeCo in IoT devices.

REFERENCES

- [1] N. Wang, Y. Chen, Y. Hu, W. Lou, and Y. T. Hou, "FeCo: Boosting intrusion detection capability in IoT networks via contrastive learning," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1409–1418.
- [2] J. Lin, W. Yu, N. Zhang, X. Yang, H. Zhang, and W. Zhao, "A survey on Internet of Things: Architecture, enabling technologies, security and privacy, and applications," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1125–1142, Oct. 2017.
- [3] T. Song, R. Li, B. Mei, J. Yu, X. Xing, and X. Cheng, "A privacy preserving communication protocol for IoT applications in smart homes," *IEEE Internet Things J.*, vol. 4, no. 6, pp. 1844–1852, Dec. 2017.

- [4] B. Omoniwa, R. Hussain, M. A. Javed, S. H. Bouk, and S. A. Malik, "Fog/edge computing-based IoT (FECIoT): Architecture, applications, and research issues," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4118–4149, Jun. 2019.
- [5] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "DfIoT: A federated self-learning anomaly detection system for IoT," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 756–767.
- [6] M. Antonakakis et al., "Understanding the mirai botNet," in *Proc. 26th {USENIX} Secur. Symp.*, 2017, pp. 1093–1110.
- [7] D. Oh, D. Kim, and W. W. Ro, "A malicious pattern detection engine for embedded security systems in the Internet of Things," *Sensors*, vol. 14, no. 12, pp. 24188–24211, 2014.
- [8] A. Ferdowsi and W. Saad, "Generative adversarial networks for distributed intrusion detection in the Internet of Things," in *Proc. IEEE Glob. Commun. Conf.*, 2019, pp. 1–6.
- [9] S. Raza, L. Wallgren, and T. Voigt, "SVELTE: Real-time intrusion detection in the Internet of Things," *Ad Hoc Netw.*, vol. 11, no. 8, pp. 2661–2674, 2013.
- [10] J. P. Amaral, L. M. Oliveira, J. J. Rodrigues, G. Han, and L. Shu, "Policy and network-based intrusion detection system for IPv6-enabled wireless sensor networks," in *Proc. IEEE Int. Conf. Commun.*, 2014, pp. 1796–1801.
- [11] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [12] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1, pp. 1–210, 2021.
- [13] N. H. Tran, W. Bao, A. Zomaya, M. N. Nguyen, and C. S. Hong, "Federated learning over wireless networks: Optimization model design and analysis," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1387–1395.
- [14] M. Eskandari, Z. H. Janjua, M. Vecchio, and F. Antonelli, "Passban IDS: An intelligent anomaly-based intrusion detection system for IoT edge devices," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 6882–6897, Aug. 2020.
- [15] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Comput. Secur.*, vol. 28, no. 1–2, pp. 18–28, 2009.
- [16] V. Jyothisna, R. Prasad, and K. M. Prasad, "A review of anomaly based intrusion detection systems," *Int. J. Comput. Appl.*, vol. 28, no. 7, pp. 26–35, 2011.
- [17] O. Kopuklu, J. Zheng, H. Xu, and G. Rigoll, "Driver anomaly detection: A dataset and contrastive learning approach," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2021, pp. 91–100.
- [18] H. H. Pajouh, R. Javidan, R. Khayami, A. Dehghantanha, and K.-K. R. Choo, "A two-layer dimension reduction and two-tier classification model for anomaly-based intrusion detection in IoT backbone networks," *IEEE Trans. Emerg. Topics Comput.*, vol. 7, no. 02, pp. 314–323, Second Quarter 2019.
- [19] N. Wang, Y. Chen, Y. Hu, W. Lou, and Y. T. Hou, "MANDA: On adversarial example detection for network intrusion detection system," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
- [20] Z. Wu, Y. Xiong, S. X. Yu, and D. Lin, "Unsupervised feature learning via non-parametric instance discrimination," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 3733–3742.
- [21] Y. Tian, D. Krishnan, and P. Isola, "Contrastive multiview coding," in *Proc. 16th Eur. Conf. Comput. Vis.*, Glasgow, U.K., Springer, 2020, pp. 776–794.
- [22] Y. Tian, C. Sun, B. Poole, D. Krishnan, C. Schmid, and P. Isola, "What makes for good views for contrastive learning?," 2020, *arXiv:2005.10243*.
- [23] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, 2009, pp. 1–6.
- [24] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," 2018, *arXiv:1802.09089*.
- [25] S. Rathore and J. H. Park, "Semi-supervised learning based distributed attack detection framework for IoT," *Appl. Soft Comput.*, vol. 72, pp. 79–89, 2018.
- [26] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Conditional variational autoencoder for prediction and feature recovery applied to intrusion detection in IoT," *Sensors*, vol. 17, no. 9, 2017, Art. no. 1967.
- [27] Y. Yang, K. Zheng, C. Wu, and Y. Yang, "Improving the classification effectiveness of intrusion detection by using improved conditional variational autoencoder and deep neural network," *Sensors*, vol. 19, no. 11, 2019, Art. no. 2528.
- [28] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. IEEE Mil. Commun. Inf. Syst. Conf.*, 2015, pp. 1–6.
- [29] H. H. Pajouh, G. Dastghaibafard, and S. Hashemi, "Two-tier network anomaly detection model: A machine learning approach," *J. Intell. Inf. Syst.*, vol. 48, no. 1, pp. 61–74, 2017.
- [30] N. Moustafa, B. Turnbull, and K.-K. R. Choo, "An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4815–4830, Jun. 2019.
- [31] R. Alshammari and A. N. Zincir-Heywood, "Can encrypted traffic be identified without port numbers, IP addresses and payload inspection?," *Comput. Netw.*, vol. 55, no. 6, pp. 1326–1350, 2011.
- [32] S. Rezvy, Y. Luo, M. Petridis, A. Lasebae, and T. Zebin, "An efficient deep learning model for intrusion classification and prediction in 5G and IoT networks," in *Proc. IEEE 53rd Annu. Conf. Inf. Sci. Syst.*, 2019, pp. 1–6.
- [33] C. Kolias, G. Kambourakis, A. Stavrou, and S. Gritzalis, "Intrusion detection in 802.11 networks: Empirical evaluation of threats and a public dataset," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 1, pp. 184–208, First Quarter 2016.
- [34] Y. Zhang et al., "Efficient and intelligent attack detection in software defined IoT networks," in *Proc. IEEE Int. Conf. Embedded Softw. Syst.*, 2020, pp. 1–9.
- [35] R. R. Fontes, S. Afzal, S. H. Brito, M. A. Santos, and C. E. Rothenberg, "Mininet-WiFi: Emulating software-defined wireless networks," in *Proc. 11th Int. Conf. Netw. Serv. Manage.*, 2015, pp. 384–389.
- [36] Y. Fan, Y. Li, M. Zhan, H. Cui, and Y. Zhang, "IoTDefender: A federated transfer learning intrusion detection framework for 5G IoT," in *Proc. IEEE 14th Int. Conf. Big Data Sci. Eng.*, 2020, pp. 88–95.
- [37] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.
- [38] H. Kang, D. H. Ahn, G. M. Lee, J. D. Yoo, K. H. Park, and H. K. Kim, "IoT network intrusion dataset," *IEEE Dataport*, 2019. [Online]. Available: <https://dx.doi.org/10.21227/q70p-q449>
- [39] N. Ravi and S. M. Shalinie, "Semisupervised-learning-based security to detect and mitigate intrusions in IoT network," *IEEE Internet Things J.*, vol. 7, no. 11, pp. 11041–11052, Nov. 2020.
- [40] A. Cosson, A. K. Sikder, L. Babun, Z. B. Celik, P. McDaniel, and A. S. Ulugac, "Sentinel: A robust intrusion detection system for IoT networks using kernel-level system information," in *Proc. Int. Conf. Internet-Things Des. Implementation*, 2021, pp. 53–66.
- [41] Y. Meidan et al., "N-baIoT—network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, Third Quarter, 2018.
- [42] Y. Liu et al., "Deep anomaly detection for time-series data in industrial IoT: A communication-efficient on-device federated learning approach," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6348–6358, Apr. 2021.
- [43] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh, A. Dehghantanha, and G. Srivastava, "Federated learning-based anomaly detection for IoT security attacks," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2545–2554, Feb. 2022.
- [44] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Secur. Privacy*, 2010, pp. 305–316.
- [45] M. Du, F. Li, G. Zheng, and V. Srikanth, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1285–1298.
- [46] Q. Yan et al., "SpecMonitor: Toward efficient passive traffic monitoring for cognitive radio networks," *IEEE Trans. Wireless Commun.*, vol. 13, no. 10, pp. 5893–5905, Oct. 2014.
- [47] Q. Yan, Y. Zheng, T. Jiang, W. Lou, and Y. T. Hou, "PeerClean: Unveiling peer-to-peer botnets through dynamic group behavior analysis," in *Proc. IEEE Conf. Comput. Commun.*, 2015, pp. 316–324.
- [48] B. Wang, Y. Zheng, W. Lou, and Y. T. Hou, "DDoS attack protection in the era of cloud computing and software-defined networking," *Comput. Netw.*, vol. 81, pp. 308–319, 2015.
- [49] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, "Network intrusion detection for IoT security based on learning techniques," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2671–2701, Third Quarter 2019.
- [50] M. A. Al-Garadi, A. Mohamed, A. K. Al-Ali, X. Du, I. Ali, and M. Guizani, "A survey of machine and deep learning methods for Internet of Things (IoT) security," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 1646–1685, Third Quarter, 2020.

- [51] S. Marchal, M. Miettinen, T. D. Nguyen, A.-R. Sadeghi, and N. Asokan, "AuDi: Toward autonomous IoT device-type identification using periodic communication," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 6, pp. 1402–1412, Jun. 2019.
- [52] R. Perdisci, T. Papastergiou, O. Alrawi, and M. Antonakakis, "IoTFinder: Efficient large-scale identification of IoT devices via passive DNS traffic analysis," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2020, pp. 474–489.
- [53] L. Yu, B. Luo, J. Ma, Z. Zhou, and Q. Liu, "You are what you broadcast: Identification of mobile and IoT devices from (public) WiFi," in *Proc. 29th {USENIX} Secur. Symp.*, 2020, pp. 55–72.
- [54] C. Spearman, *The Proof and Measurement of Association Between Two Things*. Norwalk, CT, USA: Appleton-Century-Crofts, 1961.
- [55] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 1597–1607.
- [56] T. Chen, S. Kornblith, K. Swersky, M. Norouzi, and G. E. Hinton, "Big self-supervised models are strong semi-supervised learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 22243–22255.
- [57] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9726–9735.
- [58] A. Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 8026–8037.
- [59] V. Hajisalem and S. Babaie, "A hybrid intrusion detection system based on ABC-AFS algorithm for misuse and anomaly detection," *Comput. Netw.*, vol. 136, pp. 37–50, 2018.
- [60] O. Al-Jarrah, A. Siddiqui, M. Elsalamouny, P. D. Yoo, S. Muhaidat, and K. Kim, "Machine-learning-based feature selection techniques for large-scale network intrusion detection," in *Proc. IEEE 34th Int. Conf. Distrib. Comput. Syst. Workshops*, 2014, pp. 177–181.
- [61] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.



Yimin Chen (Member, IEEE) received the BS degree in electrical engineering from Peking University, in 2010, and the PhD degree from Arizona State University, in 2018 with a focus on security and privacy in mobile computing. After completing postdoctoral research with Virginia Tech, he joined the Miner School of Computer and Information Sciences, University of Massachusetts Lowell (UMASS Lowell) as an assistant professor. Currently, his work focuses on the understanding and development of secure and privacy-aware machine learning models.



Wenjing Lou (Fellow, IEEE) is the W. C. english endowed professor of computer science with Virginia Tech and a fellow of the ACM. Her research interests cover many topics in the cybersecurity field, with her current research focusing on wireless network security, trustworthy AI, blockchain, and security and privacy problems in the Internet of Things (IoT) systems. She is a highly cited researcher by the Web of Science Group. She received the Virginia Tech Alumni Award for Research Excellence, in 2018. She received the INFOCOM Test-of-Time paper award, in 2020. She was the TPC chair for IEEE INFOCOM 2019 and ACM WiSec 2020. She was the Steering Committee Chair for IEEE CNS conference from 2013 to 2020. She is currently the vice chair of IEEE INFOCOM steering committee. She served as a program director with the US National Science Foundation (NSF) from 2014 to 2017.



Ning Wang (Member, IEEE) received the BE degree in communication engineering and the MS degree in electronics and communication engineering from the Beijing University of Posts and Telecommunications, Beijing, China, in 2015 and 2018, respectively, and the PhD degree in computer engineering from Virginia Tech, in 2023. She is an assistant professor with the Department of Computer Science and Engineer, University of South Florida. Her current research interests include federated learning, anomaly detection, adversarial machine learning, differential privacy, and LLM applications in cybersecurity.



Y. Thomas Hou (Fellow, IEEE) received the PhD degree from the NYU Tandon School of Engineering, in 1998. He is currently Bradley distinguished professor of electrical and computer engineering with Virginia Tech, Blacksburg, VA, USA, which he joined in 2002. He was a member of Research Staff with the Fujitsu Laboratories of America in Sunnyvale, CA from 1997 to 2002. His current research focuses on developing real-time optimal solutions to complex science and engineering problems arising from wireless and mobile networks. He is also interested in wireless security. He has published more than 350 papers in *IEEE/ACM journals and conferences*. His papers were recognized by 12 best paper awards from IEEE and ACM, including an IEEE INFOCOM Test of Time Paper Award in 2023. He holds six U.S. patents. He authored/co-authored two graduate textbooks. Prof. Hou was named an IEEE fellow for contributions to modeling and optimization of wireless networks. He was/is on the editorial boards of a number of IEEE and ACM transactions and journals. He was Steering committee chair of IEEE INFOCOM conference and was a member of the IEEE Communications Society Board of Governors. He was also a distinguished lecturer of the IEEE Communications Society.



Shanghao Shi (Student Member, IEEE) received the BS degree in telecommunication engineering from the Beijing University of Posts and Telecommunications. He is currently working toward the PhD degree in computer science with Virginia Tech, supervised by Prof. Wenjing Lou and Prof. Yi Shi. His research interests lie in wireless network security, CPS security, and machine learning security and privacy.